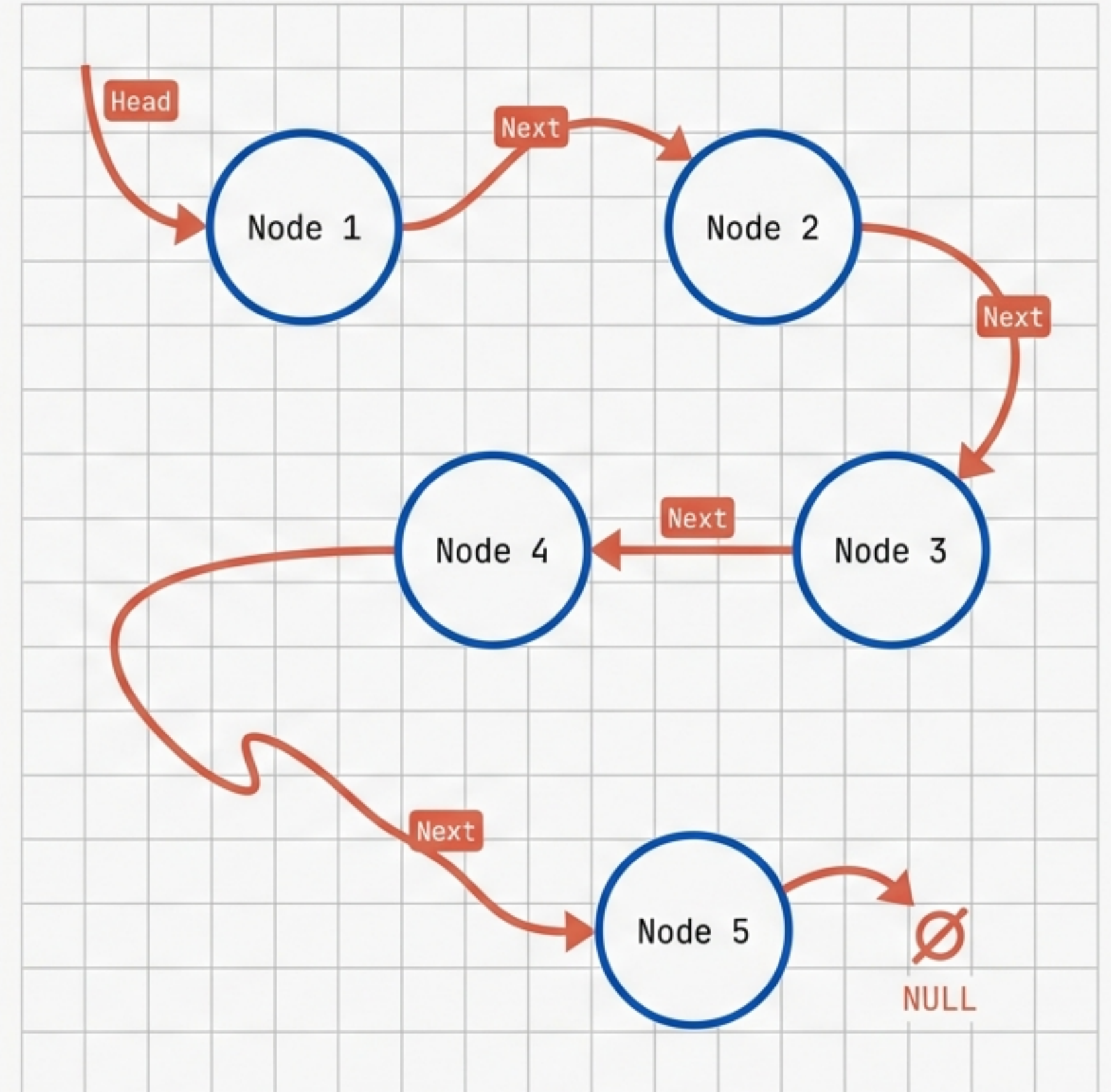


Mastering Linked Lists: DSA From Scratch

A visual guide to dynamic memory, node architecture, and algorithmic logic.

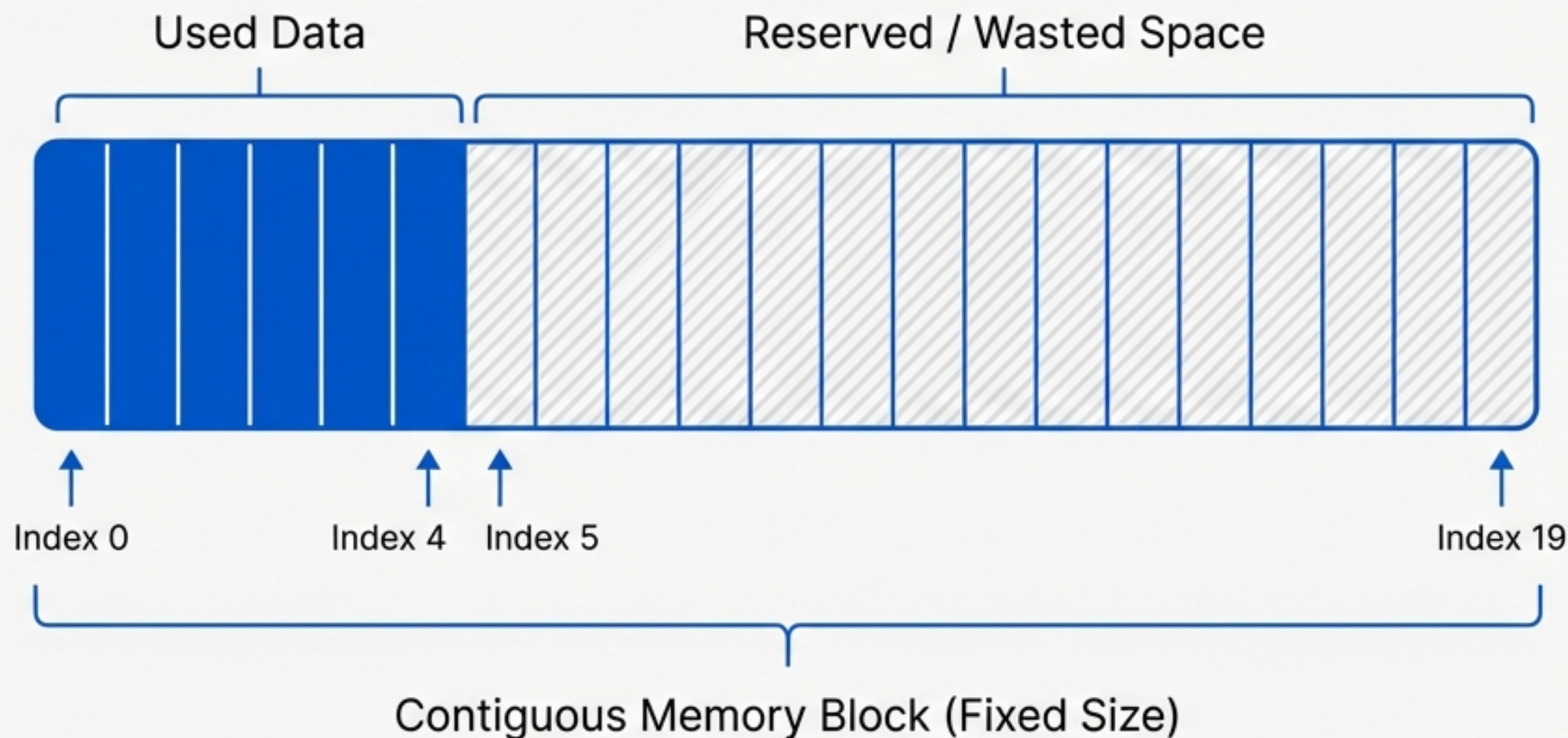


Based on the lecture "Data Structures and Algorithms From Scratch".

The Limitation of Static Data: Arrays

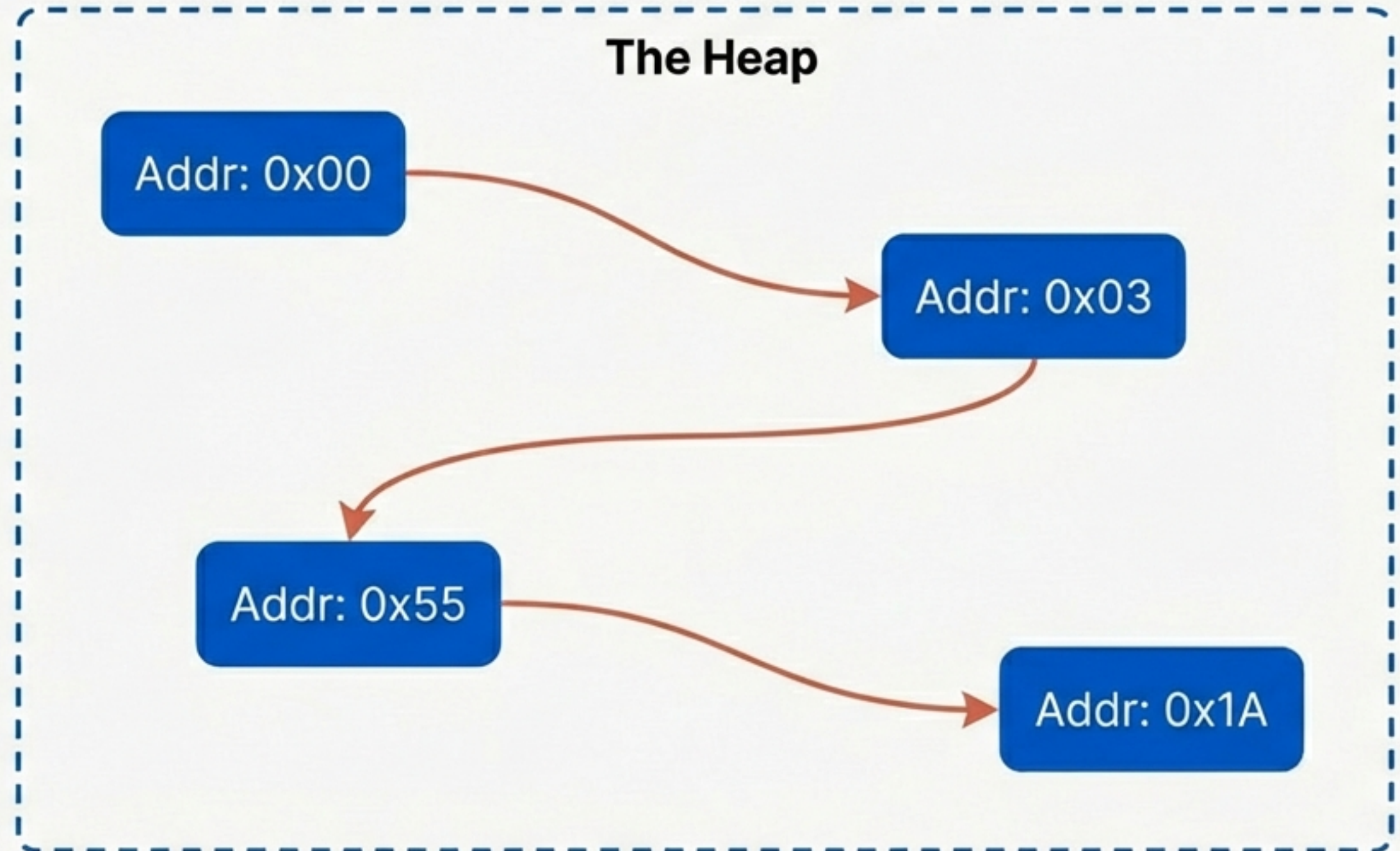
Why change?

- **Contiguous Allocation:**
Arrays occupy a single, unbroken block of memory.
- **The Waste Problem:**
Initializing size 100 but using 5 consumes all 100 units.
- **The Rigid Limit:** Cannot exceed initialized size without creating a new array.

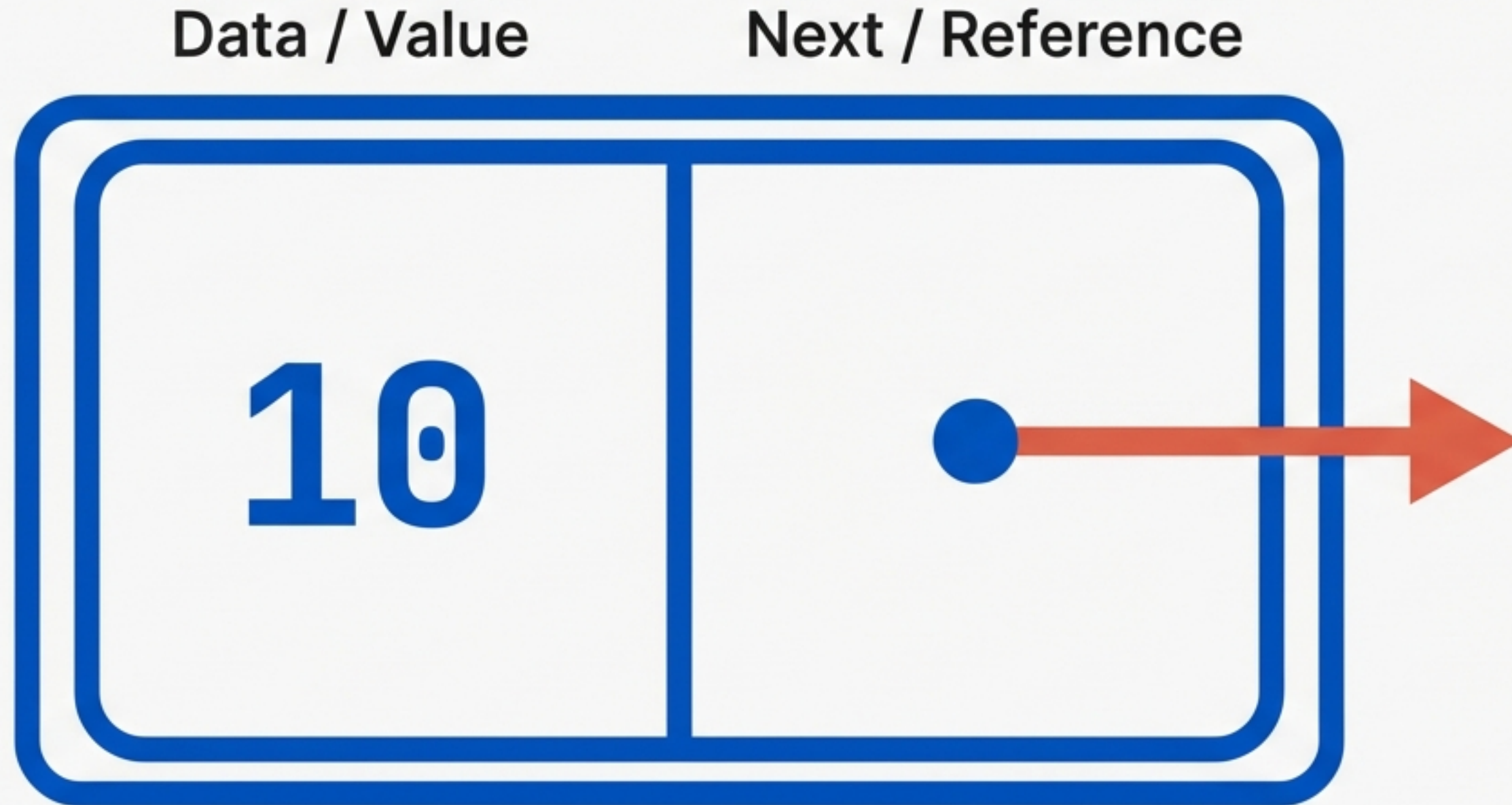


The Solution: Dynamic Memory in the Heap

- **Non-Contiguous:** Data exists wherever space is available.
- **Dynamic:** Grows and shrinks instantly. No pre-initialization.
- **The Trade-off:** We need a map (pointers) to find the next piece of data.



The Atom: Anatomy of a Node



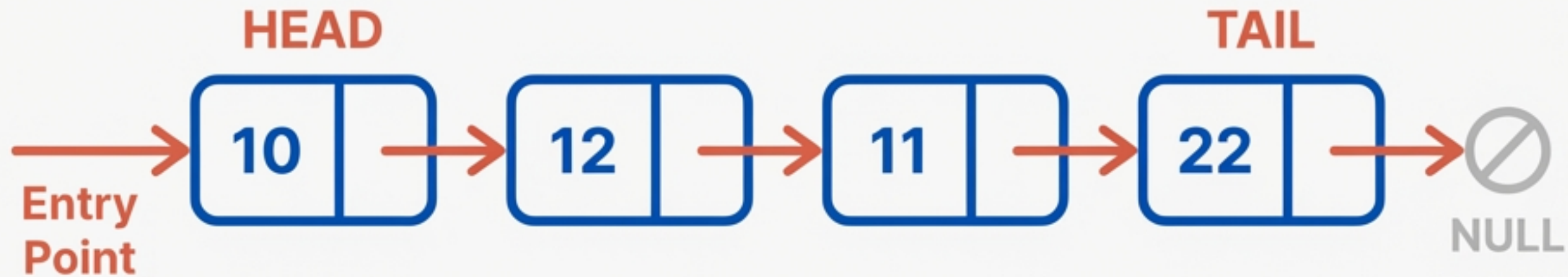
Internal Structure

Two distinct fields:

1. **Value:** The actual data we want to store.
2. **Next:** A reference holding the memory address of the subsequent node.

NOTE: These are references, not C++ pointers.

The Architecture: Head, Tail, and The Void

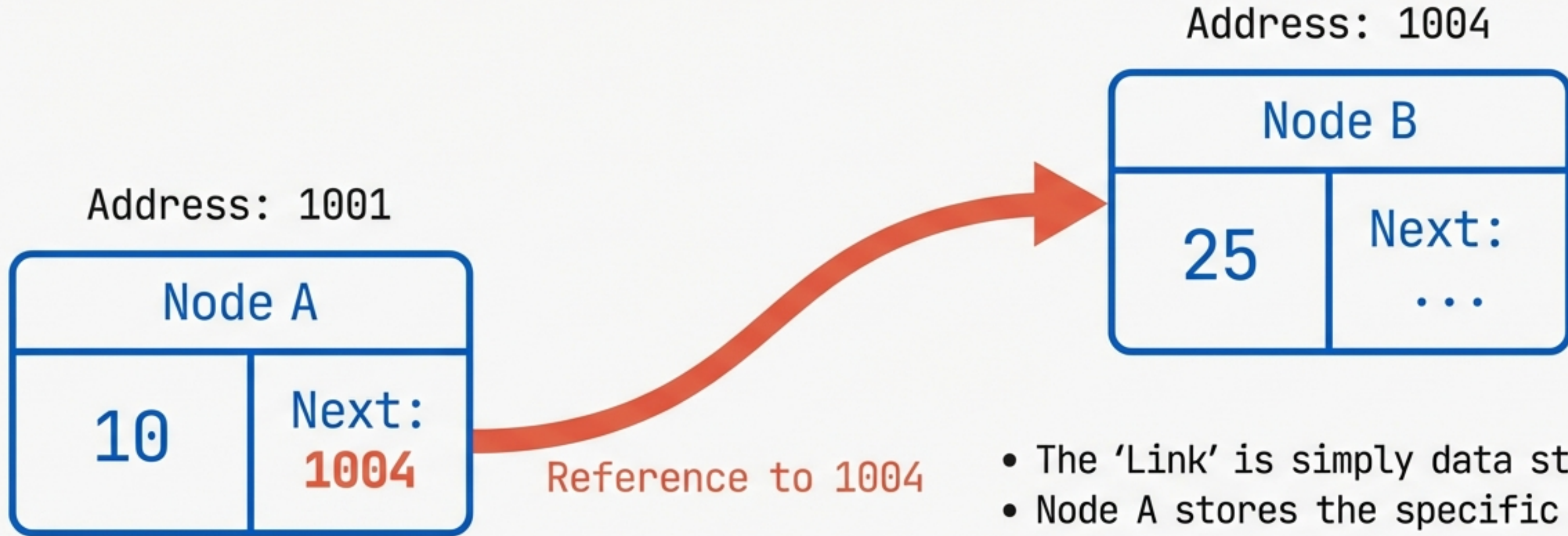


Head: The representative of the list. If you lose the Head, you lose the list.

Tail: The last node.

Null: The void. Indicates the end of the chain.

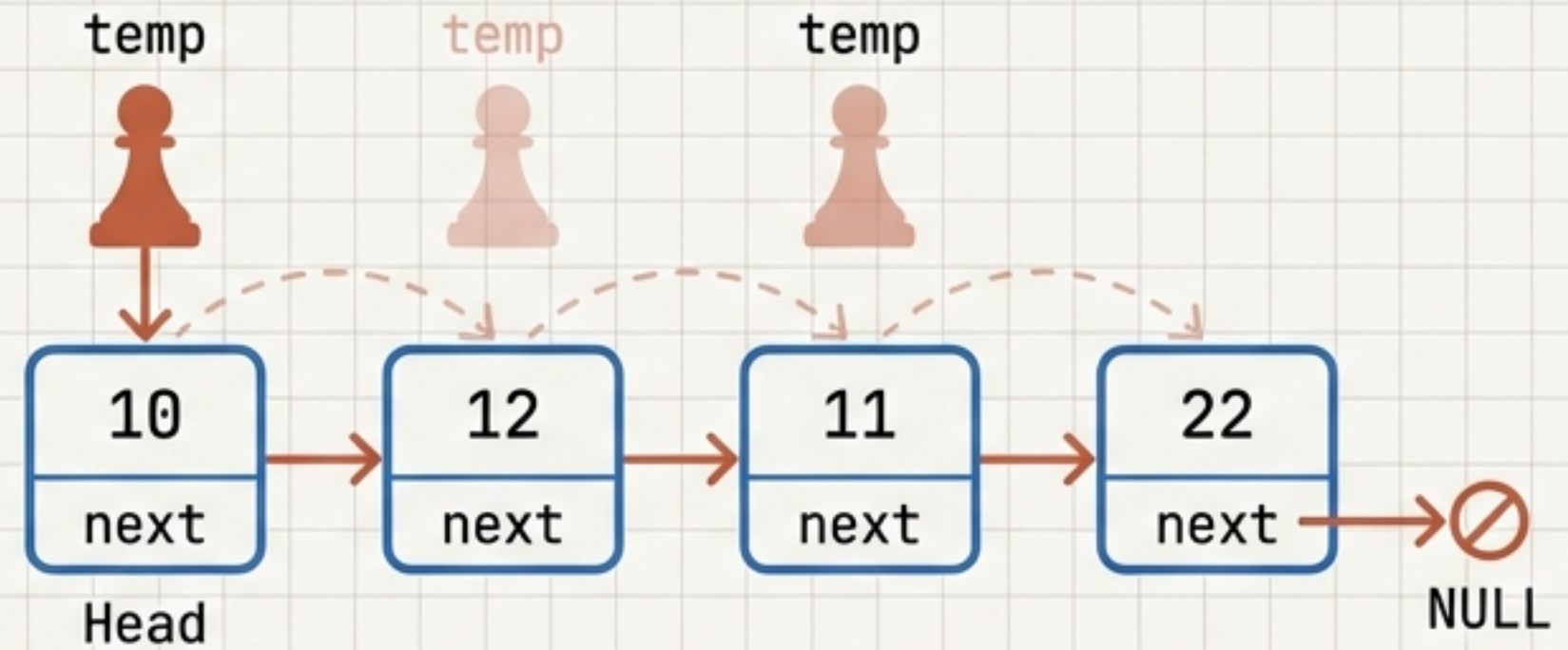
Under the Hood: Addressing and Mapping



- The 'Link' is simply data storage.
- Node A stores the specific memory location of Node B.
- The computer uses this address to "jump" across the Heap.

Traversal: The 'Traveler' Metaphor

```
Node temp = head;  
while (temp != null) {  
    print(temp.value);  
    temp = temp.next; // Move to next address  
}
```

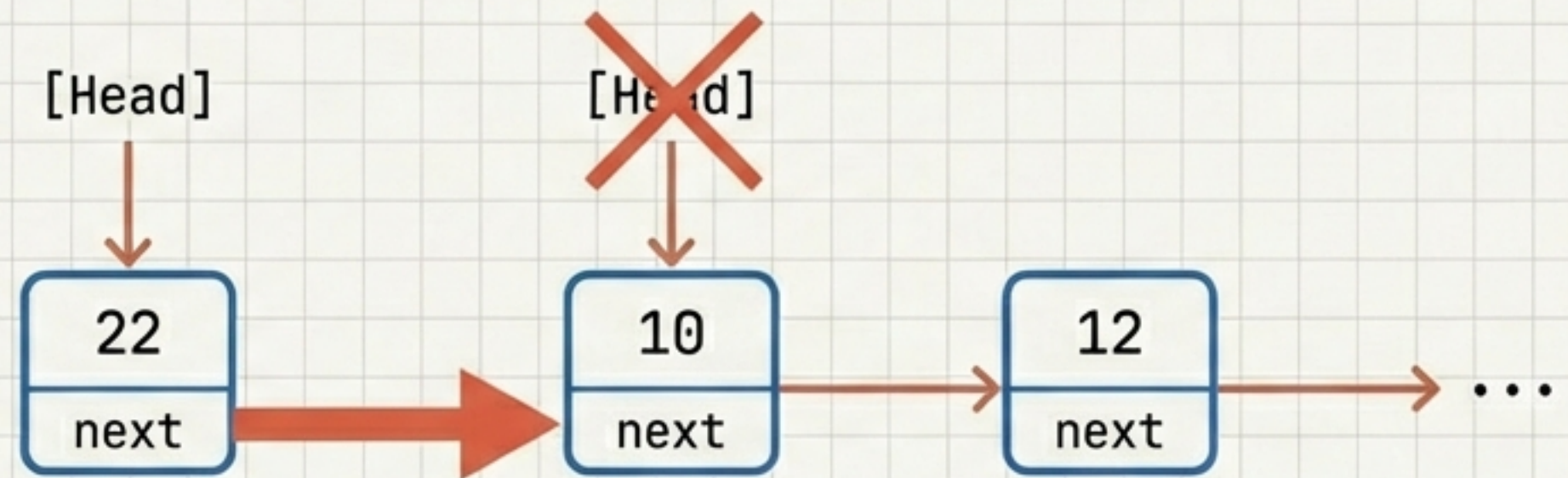


Key Insight: Sequential Access Only. To access element #3, you must visit #1 and #2 first.

Operation: Insert at First (Head)

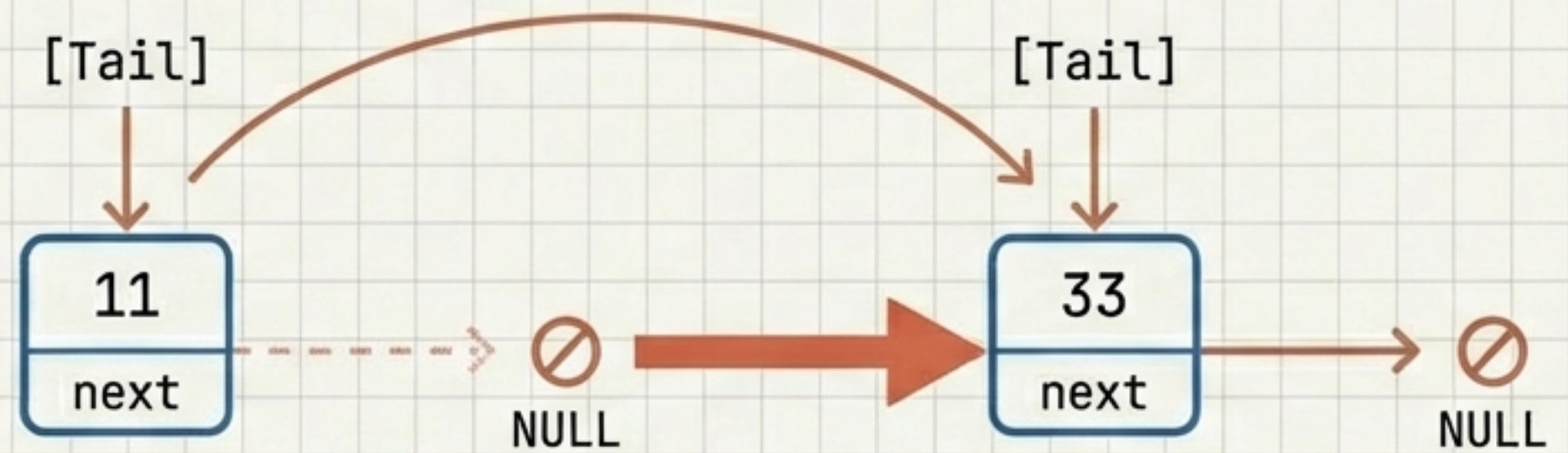
```
NewNode.next = head;  
head = NewNode;  
size++;
```

The new node becomes the new entry point.



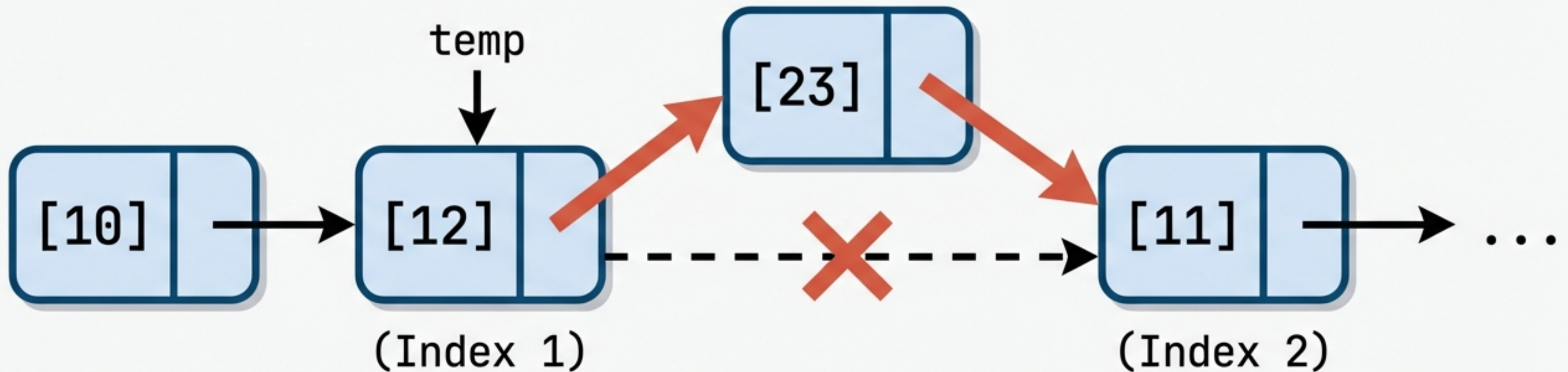
Operation: Insert at Last (Tail)

```
tail.next = node;  
tail = node;  
node.next = null;  
size++;
```



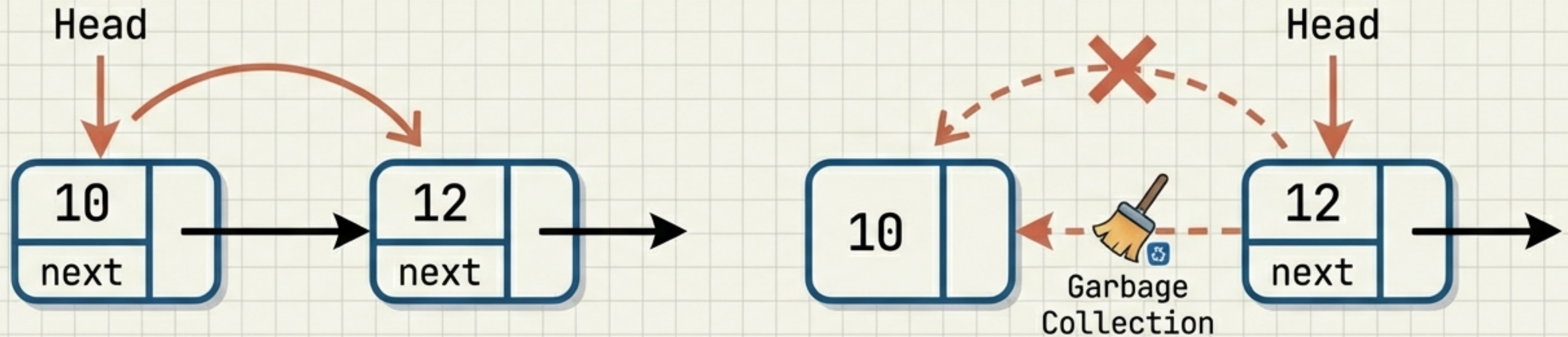
Operation: Insert at Index (The Middle)

Scenario: Insert Value 23 between Index 1 and 2.



Order Matters! Connect the New Node *before* breaking the old link to avoid losing the rest of the list.

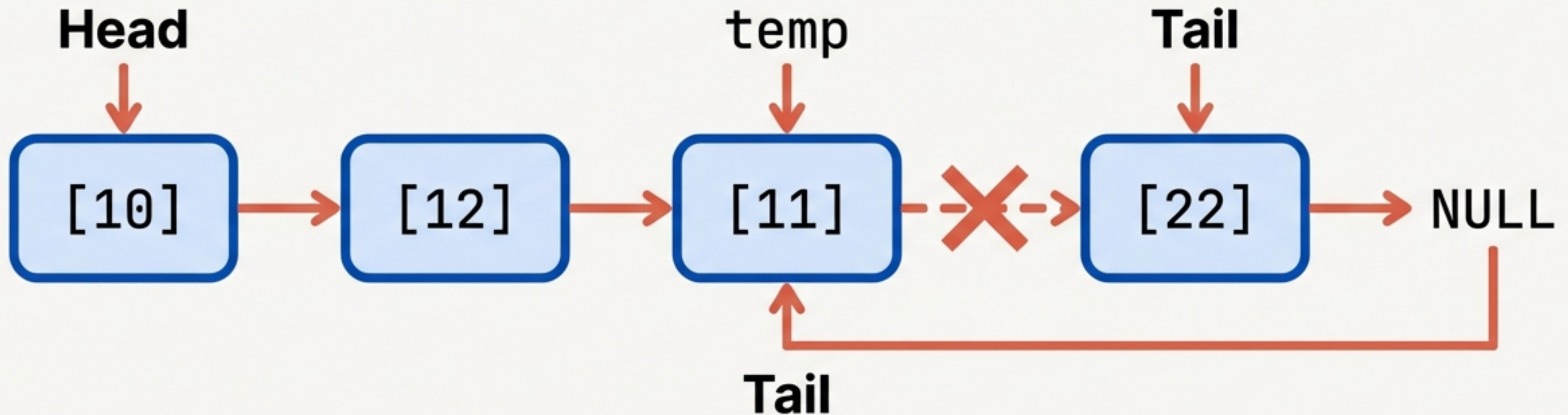
Deletion: Removing the Head



- **Orphaned Node:** No references point to the old head.
- **Garbage Collection:** The OS automatically sweeps unreferenced nodes to free memory.

```
head = head.next;  
size--;
```


Deletion: Removing the Tail



Logic

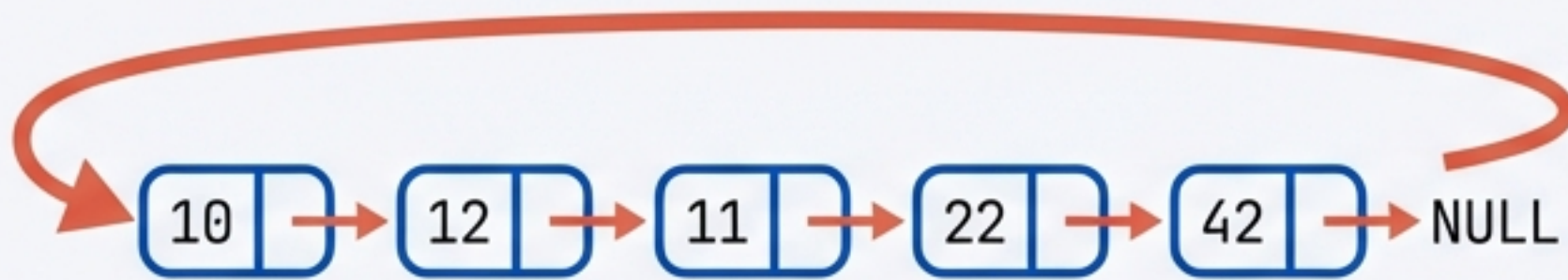
- **The Problem:** We cannot go backwards from Tail.
- **The Fix:** Traverse from Head.
- **Stop Condition:** `while (temp.next != tail)`

Code

```
temp.next = null;  
tail = temp;  
size--;
```


Optimization: The 'Size' Strategy

The Slow Way



Iterative Count: $O(n)$.
Must traverse entire list to
count nodes.

The Fast Way



int size
5

Variable Maintenance: $O(1)$.
Increment `size++` on insert,
decrement `size--` on delete.

Takeaway: Maintain a state variable for instant access to list size.

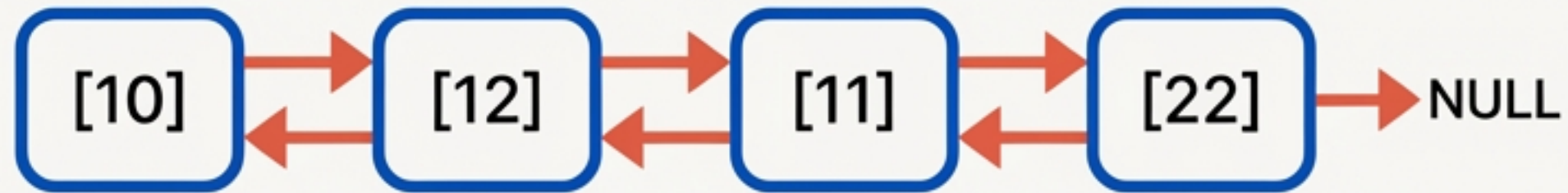
Beyond the Basics: Types of Lists

Singly Linked



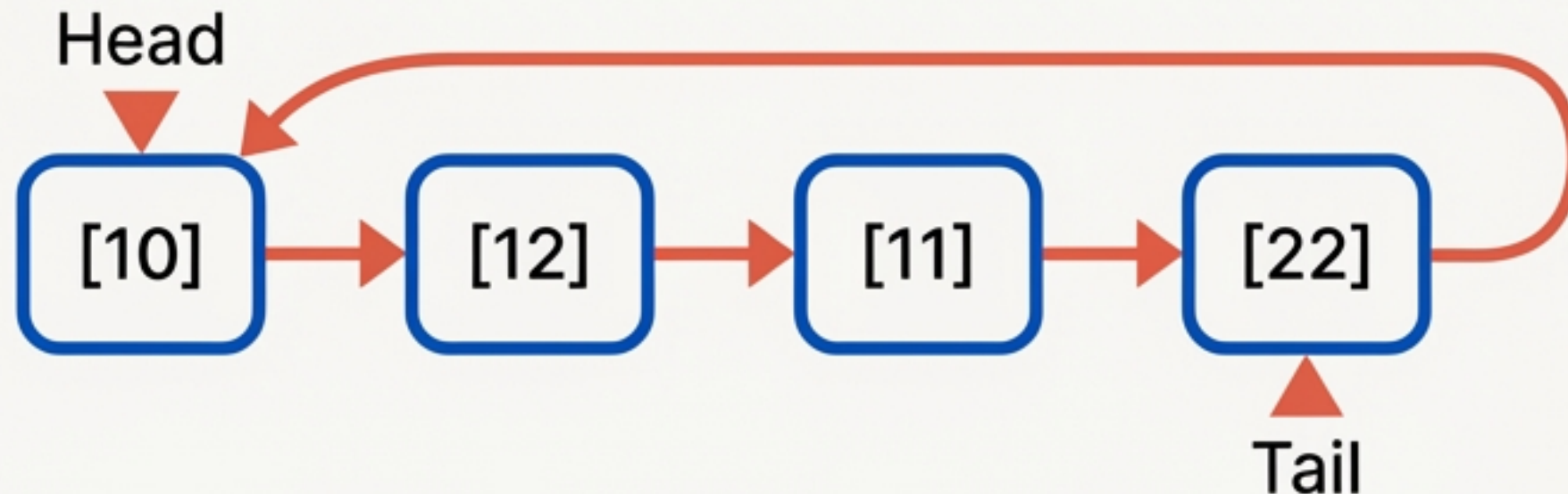
Singly: One-way traffic. Efficient memory.

Doubly Linked



Doubly: Two-way traffic. Easier traversal, more memory.

Circular

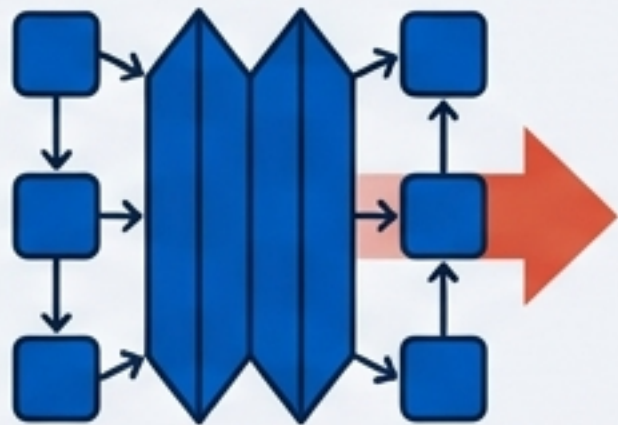


Circular: Infinite loop. Tail connects to Head.

Key Takeaways

Dynamic Sizing

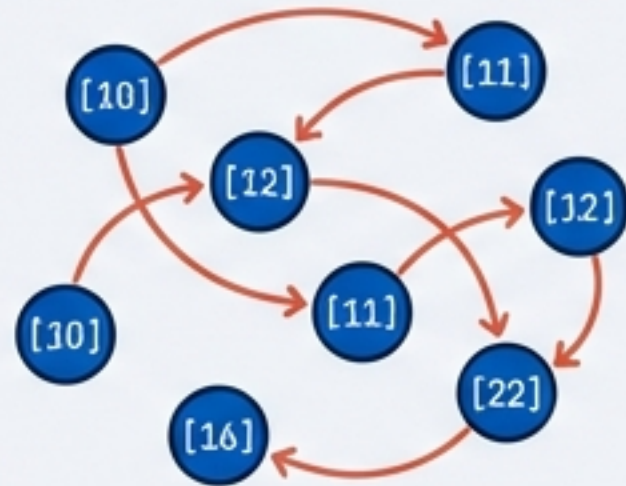
JetBrains Mono Bold



Memory efficient;
consumes exactly
what is needed.

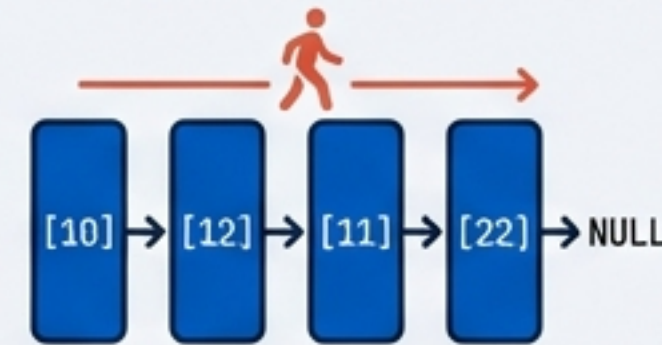
Non-Contiguous

JetBrains Mono Bold



Utilizes Heap memory;
no contiguous block
required.

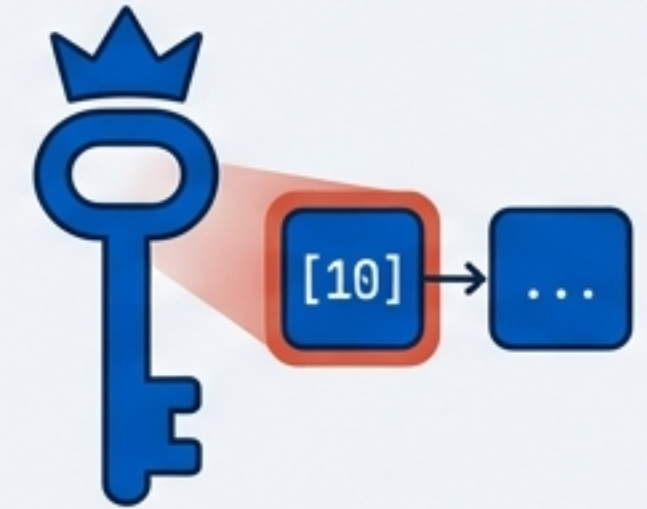
Sequential Access



Must traverse via
pointers; no random
access.

The Head

JetBrains Mono Bold



The critical entry
point. Lose the head,
lose the list.

Linked Lists are defined by their operations—drawing the pointers is the best way to understand the code.